e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

MICROSERVICES ARCHITECTURE

Gaurav Prabhakar[*1]

[*1]Senior Software Engineer | Application Architect, Melissa, Tx

DOI: https://www.doi.org/10.56726/IRJMETS66464

ABSTRACT

This paper provides a comprehensive overview of microservices architecture, focusing on its implementation. We discuss the core principles, key components, deployment considerations, challenges, and best practices for building robust and scalable microservices-based applications with C#. The paper emphasizes the benefits of adopting microservices, such as improved scalability, agility, and maintainability, while also addressing common challenges like distributed system complexity and data management. Specific examples and code snippets illustrate the practical aspects of implementing microservices with ASP.NET Core and related technologies.

Keywords: Microservices, C#, Web Api, Docker Container.

I. INTRODUCTION

Microservices architecture represents a contemporary paradigm in software development, where applications are designed as assemblies of small, independent services, each tasked with a specific function. This approach empowers organizations to boost scalability, maintainability, and deployment efficiency. Numerous leading enterprises have embraced this architecture to refine their IT operations and drive swift expansion. Each service is dedicated to a particular business capability and interacts with other services through well-defined APIs. This is in stark contrast to monolithic architectures, where all functionalities are tightly integrated within a single application.

II. MICROSERVICES ARCHITECTURE

Microservices is an architectural style where applications are built as a collection of small, autonomous services that are designed to fulfill specific business functions. These services work together to form the complete application but operate independently of each other. This modular approach provides several advantages, including improved scalability, easier maintenance, and faster deployment cycles.

Key Characteristics:

1. **Independence**: Each microservice can be developed, deployed, and scaled independently, without affecting other services.
2. **Decentralization**: Microservices encourage decentralization of data management, often using different databases for different services.
3. **Technology Diversity**: Different services can be developed using different programming languages, databases, and frameworks.
4. **Scalability**: Microservices allow for selective scaling. Individual services can be scaled based on their specific demand.
5. **Fault Isolation**: Failure in one service does not affect the entire system, making it more resilient.

Components of Microservices Architecture

1. **Service Components**:
   - **Business Logic Services**: Each microservice addresses a specific business capability. For example, an e-commerce application might have separate services for inventory management, order processing, and payment.
   - **Data Services**: These manage data storage and retrieval for each microservice. Each service often has its own database to ensure independence.
2. **API Gateway**:
   - An API gateway acts as a single entry point for external clients to access the microservices. It handles routing requests to the appropriate services, authentication, authorization, and potentially other cross-cutting concerns like rate limiting. In a C# microservices architecture, Ocelot or similar gateway solutions are frequently used.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

3. **Service Discovery**:

- Service discovery mechanisms are essential for microservices to dynamically locate and communicate with each other in a distributed system. These mechanisms are crucial in environments where the location of services can change frequently. Service discovery can be implemented in two primary ways: client-side discovery and server-side discovery. In client-side discovery, the client queries a service registry to find the desired service instance. In server-side discovery, the API Gateway handles the request by querying the service registry and forwarding it to the appropriate service. Common tools used for service discovery include DNS, Consul, and etcd. Moreover, .NET offers libraries that integrate seamlessly with these service discovery tools, making it easier for developers to implement robust and efficient service discovery within their microservices architecture.

4. **Inter-Service Communication:**

- **REST APIs (Representational State Transfer):** A common approach where services exchange data through HTTP requests. ASP.NET Core provides excellent support for building RESTful APIs.
- **Message Queues (e.g., RabbitMQ, Azure Service Bus):** Asynchronous communication where services send messages to a queue, allowing for decoupling and improved resilience.
- **Event Streaming (e.g., Kafka):** Services publish events to a stream, and other services subscribe to relevant events. This supports real-time data processing and reactive architectures.

5. **Database Management**:

- **Dedicated Databases per Service**: Each microservice has its own database, promoting autonomy and simplifying data management.
- **Data Replication:** Data can be replicated across multiple databases for high availability and fault tolerance.
- **Event Sourcing:** Changes are recorded as a sequence of events, providing a history of the data and enabling easier auditing and recovery.
- **Data Consistency**: Maintaining data consistency across multiple services, especially in scenarios involving distributed transactions, can be challenging. Strategies like sagas or two-phase commit can be used, but careful consideration is needed.

6. **Containerization**:

- Microservices are often deployed in containers, such as those provided by Docker, to ensure consistency across various environments and to promote efficient resource utilization. Containerization with Docker packages each microservice along with its dependencies into a self-contained unit. This approach not only ensures consistency across different environments but also simplifies deployment and enhances portability.

7. **Orchestration Platforms:**

- Platforms like Kubernetes manage the lifecycle of containers, including deployment, scaling, load balancing, and health monitoring. They automate many of the complex tasks involved in running microservices in a production environment.

8. **DevOps and CI/CD**:

- Continuous Integration and Continuous Deployment pipelines automate the process of building, testing, and deploying microservices, ensuring rapid and reliable releases.

9. **Monitoring and Logging**:

- For .NET, effective tools and practices for monitoring and logging are crucial in tracking the health and performance of microservices. Prometheus is a powerful tool for monitoring, providing real-time insights into the system's performance and resource utilization. For logging and analysis, the ELK Stack (Elasticsearch, Logstash, Kibana) is highly effective. Elasticsearch enables efficient storage and search capabilities, Logstash processes and transports logs, and Kibana offers a user-friendly interface for visualizing data and logs.

Core Principles of Microservices Architecture

- **Single Responsibility Principle:** Each microservice should have a single, well-defined responsibility. This promotes modularity, reduces complexity, and enhances maintainability.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

- **Loose Coupling:** Services should interact through standardized interfaces (APIs) with minimal dependencies on the internal implementations of other services. This ensures that changes in one service do not necessitate changes in others, thus enhancing flexibility.
- **Independent Deployability:** Each microservice can be deployed independently without affecting other services. This enables faster release cycles, reduces deployment risks, and allows for more frequent updates and improvements.
- **Technology Agnosticism:** The choice of technology for each service should be independent of other services, allowing teams to use the best tools and languages for the specific tasks. This flexibility can lead to more efficient and innovative solutions.

Best Practices for Implementing Microservices:

- **Bounded Context:** Define clear boundaries for each microservice based on distinct business domain concepts. This ensures that services remain focused, reduces coupling, and avoids unnecessary complexity.
- **API Design:** Design well-defined and thoroughly documented APIs, following RESTful principles or other suitable communication patterns. Consistent API design enhances interoperability and minimizes integration challenges.
- **Gradual Migration:** When transitioning from a monolithic architecture to microservices, adopt an incremental approach. Start by identifying components suitable for refactoring and gradually decompose the monolith. This reduces risk and allows for smoother transitions.

III. BENEFITS

1. **Enhanced Agility**: Teams can develop, test, and deploy services independently, enabling faster delivery and iteration of features.
2. **Improved Scalability**: Services can be scaled independently, optimizing resource usage and performance.
3. **Resilience**: The isolation of services helps contain failures, improving the overall stability of the application. If one service fails, it doesn't necessarily bring down the entire application.
4. **Flexibility in Technology Choices**: Teams can choose the best-suited technology stack for each service without being constrained by a monolithic architecture.
5. **Easier Maintenance and Updates**: Smaller, manageable codebases make maintenance and updates less complex and less risky.

IV. CHALLENGES

1. **Complexity in Management**: Managing a distributed system with numerous services can be complex and requires sophisticated tooling.
2. **Inter-service Communication**: Ensuring reliable and efficient communication between services can be challenging.
3. **Data Consistency**: Maintaining data consistency across services requires careful planning and strategy.
4. **Deployment Overheads**: Frequent deployment of services can increase operational overhead.
5. **Security**: Securing microservices involves managing authentication and authorization across multiple services.

V. CASE STUDIES AND REAL-WORLD EXAMPLES

1. **Netflix:** By 2008, Netflix encountered frequent service outages due to scaling issues. In response, they transitioned to a microservices architecture, enhancing operational resilience and responsiveness. Adopting Amazon Web Services (AWS) as their cloud provider, Netflix gradually migrated from a monolithic architecture to a microservices-based infrastructure. This shift significantly improved their ability to handle daily API requests and transformed the way viewers consumed content. By 2017, Netflix managed over 700 microservices, setting a benchmark for modernizing IT operations and addressing large-scale challenges.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

2. Amazon: Amazon uses microservices to support its vast array of services and products, allowing for independent scaling and updates. In the early 2000s, Amazon recognized the limitations of its monolithic application design, which made upgrades cumbersome and slow, often resulting in significant downtime and costs. To address these challenges, Amazon decomposed its monolithic architecture into small, independent services that could function autonomously. This shift allowed for faster feature releases and enhancements, substantially improving site functionality and user experience. By employing microservices, Amazon scaled efficiently and became one of the most valuable companies worldwide, with a market cap of approximately $1.6 trillion by August 2022.

3. Uber: Uber leveraged microservices to manage different aspects of its platform, such as ride requests, payments, and user management. Shortly after its launch, Uber's original monolithic architecture created bottlenecks that impeded rapid feature development and bug fixes. Recognizing these challenges, Uber transitioned to developing separate microservices for various platform components. This move enabled more efficient scaling, faster feature deployment, and improved system reliability.

VI. EXAMPLE OF CODE BASED SOLUTION

Let's walk through the solution architecture for a microservices-based application using .NET and C#. We'll cover the following steps to ensure a robust and scalable architecture:

1. Define Business Domains

Identify and define key business domains to determine the boundaries for each microservice. For example:

- **Product Service:** Manages product catalog and inventory.
- **Order Service:** Handles order processing.
- **Payment Service:** Manages payment processing.
- **User Service:** Manages user accounts and authentication.

2. Design Microservices

For each business domain, design individual microservices using .NET. Each microservice will have its own database and will expose APIs for communication.

3. API Gateway

Implement an API Gateway using Ocelot to act as the single entry point for client requests. The API Gateway will route requests to appropriate microservices, handle authentication, rate limiting, and response aggregation.

4. Service Discovery

Use Consul or Eureka for service discovery to enable microservices to dynamically locate each other. Integrate the service discovery mechanism with .NET microservices.

5. Communication

Choose appropriate communication protocols:

- **Synchronous Communication:** Use RESTful APIs with ASP.NET Core for real-time communication.
- **Asynchronous Communication:** Use message brokers like RabbitMQ or Azure Service Bus for event-driven communication.

6. Data Management

Design a database strategy that includes:

- **Dedicated Databases per Service:** Each microservice has its own database, promoting autonomy and simplifying data management.
- **Event Sourcing:** Record changes as events to maintain a history of data changes.
- **Data Replication:** Ensure high availability and fault tolerance through data replication.

7. Containerization

Use Docker to containerize microservices. Each microservice, along with its dependencies, will be packaged into a Docker container.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

8. Deployment and Orchestration

Use Kubernetes for orchestrating and managing Docker containers. Set up CI/CD pipelines with Azure DevOps to automate the build, test, and deployment processes.

9. Monitoring and Logging

Implement monitoring and logging tools:

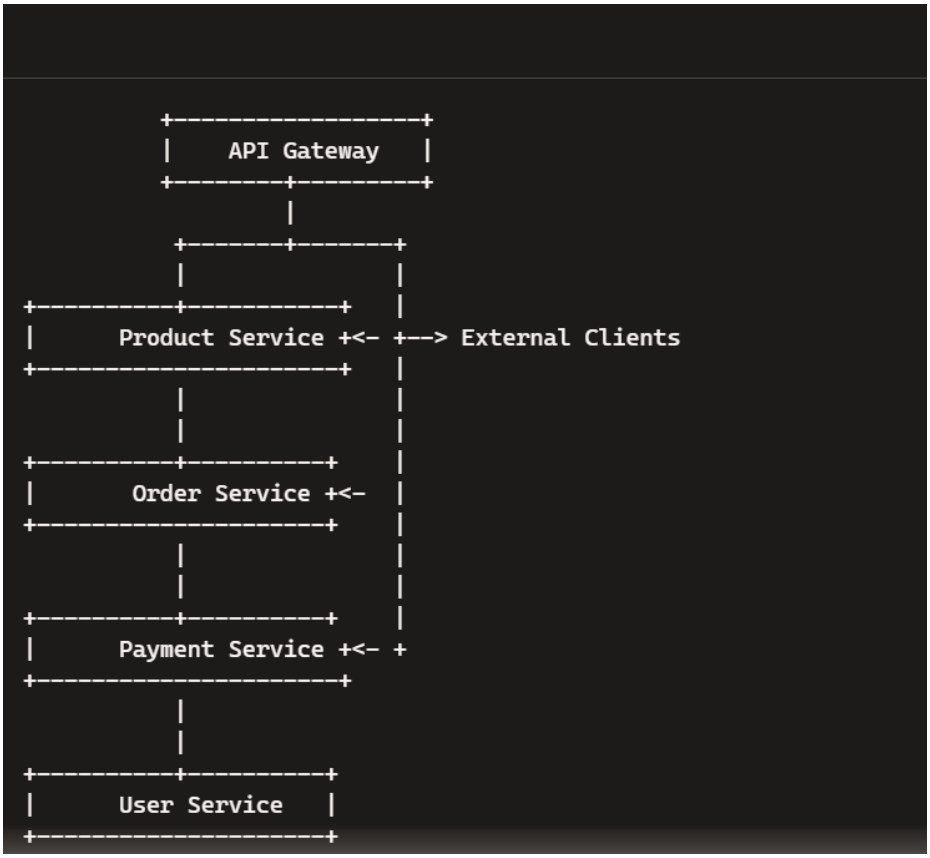
- **Monitoring:** Use Prometheus and Grafana for real-time monitoring.
- **Logging:** Use ELK Stack (Elasticsearch, Logstash, Kibana) for logging and analysis.

10. Security

Ensure security at multiple layers:

- **Authentication and Authorization:** Use OAuth2 and JWT for secure access.
- **Data Encryption:** Encrypt data at rest and in transit.
- **Vulnerability Scanning:** Regularly scan microservices and containers for vulnerabilities.

Sample Architecture Diagram:



Example Code for Product Service:-

Here's a basic example of a Product Service microservice using C# and ASP.NET Core:

1. **Create a new ASP.NET Core Web API project:**
   - Open your terminal and run the following command:

```
dotnet new webapi -n ProductService
cd ProductService
```

2. **Define the Product model:**
   - Create a new file Models/Product.cs:

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

```
Csharp

namespace ProductService.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

3. **Create a Product repository:**

- Create a new file Repositories/ProductRepository.cs:

```
using ProductService.Models;
using System.Collections.Generic;
using System.Linq;

namespace ProductService.Repositories
{
    public class ProductRepository : IProductRepository
    {
        private readonly List<Product> _products = new List<Product>
        {
            new Product { Id = 1, Name = "Product 1", Price = 10.0m },
            new Product { Id = 2, Name = "Product 2", Price = 20.0m }
        };

        public IEnumerable<Product> GetAll()
        {
            return _products;
        }

        public Product GetById(int id)
        {
            return _products.FirstOrDefault(p => p.Id == id);
        }

        public void Add(Product product)
        {
            _products.Add(product);
        }

        public void Update(Product product)
        {
            var existingProduct = GetById(product.Id);
            if (existingProduct != null)
            {
                existingProduct.Name = product.Name;
                existingProduct.Price = product.Price;
            }
        }

        public void Delete(int id)
        {
            var product = GetById(id);
            if (product != null)
            {
                _products.Remove(product);
            }
        }
    }
}
```

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

4. **Create a Product repository interface:**

- Create a new file Repositories/IProductRepository.cs:

```
using ProductService.Models;
using System.Collections.Generic;

namespace ProductService.Repositories
{
    public interface IProductRepository
    {
        IEnumerable<Product> GetAll();
        Product GetById(int id);
        void Add(Product product);
        void Update(Product product);
        void Delete(int id);
    }
}
```

5. **Create a Product controller:**

- Create a new file Controllers/ProductController.cs:

```
using Microsoft.AspNetCore.Mvc;
using ProductService.Models;
using ProductService.Repositories;
using System.Collections.Generic;

namespace ProductService.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController : ControllerBase
    {
        private readonly IProductRepository _repository;

        public ProductController(IProductRepository repository)
        {
            _repository = repository;
        }

        [HttpGet]
        public ActionResult<IEnumerable<Product>> GetAll()
        {
            return Ok(_repository.GetAll());
        }

        [HttpGet("{id}")]
        public ActionResult<Product> GetById(int id)
        {
            var product = _repository.GetById(id);
            if (product == null)
            {
                return NotFound();
            }

            return Ok(product);
        }

        [HttpPost]
        public ActionResult Add(Product product)
        {
            _repository.Add(product);
            return CreatedAtAction(nameof(GetById), new { id = product.Id
        }

        [HttpPut("{id}")]
        public ActionResult Update(int id, Product product)
        {
            if (id != product.Id)
            {
                return BadRequest();
            }

            _repository.Update(product);
            return NoContent();
        }

        [HttpDelete("{id}")]
        public ActionResult Delete(int id)
        {
            _repository.Delete(id);
            return NoContent();
        }
    }
}
```

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

6. **Register services in** Startup.cs**:**

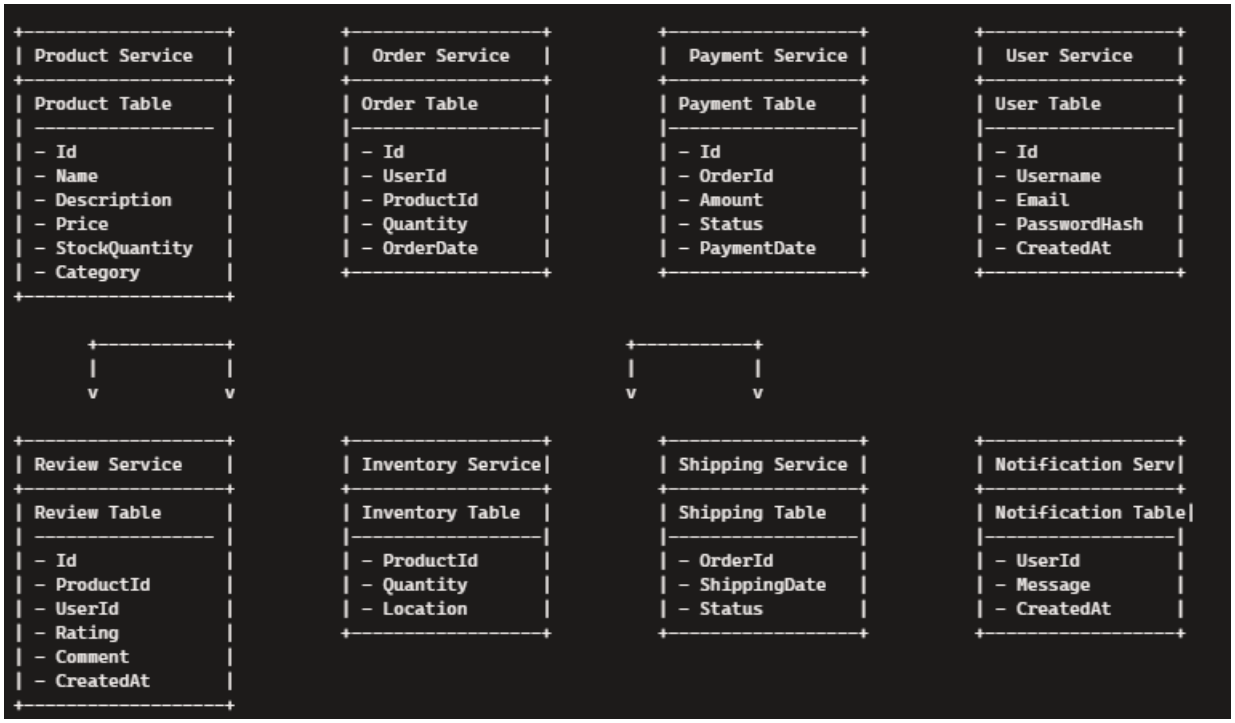
- Modify the Startup.cs file to register the product repository:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllers();
    services.AddSingleton<IProductRepository, ProductRepository>();
}
```

7. **Run the application:**

- Run your application using the following command: dotnet run

Database Diagram:



Description of Components:

1. **Product Service:**

- **Product Table:** Manages details of products, including Id, Name, Description, Price, StockQuantity, and Category.

2. **Order Service:**

- **Order Table:** Manages orders, including Id, UserId, ProductId, Quantity, and OrderDate.

3. **Payment Service:**

- **Payment Table:** Manages payments, including Id, OrderId, Amount, Status, and PaymentDate.

4. **User Service:**

- **User Table:** Manages user details, including Id, Username, Email, PasswordHash, and CreatedAt.

Additional Services:

5. **Review Service:**

- **Review Table:** Manages product reviews, including Id, ProductId, UserId, Rating, Comment, and CreatedAt.

6. **Inventory Service:**

- **Inventory Table:** Manages inventory details, including ProductId, Quantity, and Location.

7. **Shipping Service:**

- **Shipping Table**: Manages shipping details, including OrderId, ShippingDate, and Status.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

8. **Notification Service:**
   - **Notification Table:** Manages notifications, including UserId, Message, and CreatedAt.

This comprehensive view ensures all critical aspects of the application's functionality are managed effectively by individual microservices. Each service is autonomous and can independently scale and deploy.

This diagram is a simplified conceptual model. In a real-world application, you may have additional tables, fields, and relationships to accommodate more complex business requirements.

Deployment/ Containerization :

1. **Define Projects and Initialize Git Repository**
   - Create separate .NET projects for each microservice and initialize a Git repository.

```
dotnet new webapi -n ProductService
cd ProductService
git init
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools

# Repeat for other services
dotnet new webapi -n OrderService
dotnet new webapi -n PaymentService
dotnet new webapi -n UserService
```

2. **Commit Initial Code**
   - Build and commit source code to the Git repository.
3. **Set Up Continuous Integration/Continuous Deployment (CI/CD)**
   - Set up CI/CD pipelines using GitHub Actions, GitLab CI, or Azure DevOps.
4. **GitHub Actions Workflow**
   - Create a .github/workflows/ci-cd.yml file in your repository.

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - master
  pull_request:
    branches:
      - master

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v2

    - name: Set up .NET
      uses: actions/setup-dotnet@v1
      with:
        dotnet-version: '7.0.x'

    - name: Restore dependencies
      run: dotnet restore

    - name: Build
      run: dotnet build --configuration Release --no-restore

    - name: Run tests
      run: dotnet test --no-restore --verbosity normal

    - name: Publish
      run: dotnet publish --configuration Release --output ./publish --no-restore

    - name: Build Docker image
      run: docker build -t productservice:latest .

    - name: Push Docker image to registry
      run: |
        echo "${{ secrets.DOCKER_PASSWORD }}" | docker login -u "${{ secrets.DOCKER_USERNAME }}" --password-stdin
        docker tag productservice:latest your-docker-username/productservice:latest
        docker push your-docker-username/productservice:latest
```

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

5. **Dockerize Each Service**
   - Create Dockerfile for each service. Here's an example for the Product Service:

Product Service Dockerfile:

```
# Use the official .NET runtime as a parent image
FROM mcr.microsoft.com/dotnet/aspnet:7.0 AS base
WORKDIR /app
EXPOSE 80

# Use the SDK image to build and publish the app
FROM mcr.microsoft.com/dotnet/sdk:7.0 AS build
WORKDIR /src
COPY ["ProductService/ProductService.csproj", "ProductService/"]
RUN dotnet restore "ProductService/ProductService.csproj"
COPY . .
WORKDIR "/src/ProductService"
RUN dotnet build "ProductService.csproj" -c Release -o /app/build
RUN dotnet publish "ProductService.csproj" -c Release -o /app/publish

# Copy the published app to the base image and set the entry point
FROM base AS final
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "ProductService.dll"]
```

6. **Docker Compose**
   - Create a docker-compose.yml file to define and manage the deployment of all services.

```
version: '3.4'
services:
  product-service:
    image: productservice
    build:
      context: .
      dockerfile: ProductService/Dockerfile
    ports:
      - "5001:80"
  order-service:
    image: orderservice
    build:
      context: .
      dockerfile: OrderService/Dockerfile
    ports:
      - "5002:80"
  payment-service:
    image: paymentservice
    build:
      context: .
      dockerfile: PaymentService/Dockerfile
    ports:
      - "5003:80"
  user-service:
    image: userservice
    build:
      context: .
      dockerfile: UserService/Dockerfile
    ports:
      - "5004:80"
```

7. **API Gateway and Service Discovery**
   - Use Ocelot for API Gateway and Consul for service discovery.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:07/Issue:01/January-2025 Impact Factor- 8.187 www.irjmets.com

Ocelot Configuration:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/product/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [{ "Host": "product-service", "Port": 80 }],
      "UpstreamPathTemplate": "/product/{everything}",
      "UpstreamHttpMethod": [ "Get", "Post", "Put", "Delete" ]
    },
    // Repeat for other services
  ],
  "GlobalConfiguration": {
    "BaseUrl": "http://localhost:5000"
  }
}
```

8. Monitoring and Logging

- Logging: Use Serilog for structured logging.
- Monitoring: Implement monitoring with Prometheus and Grafana.

9. Kubernetes (Optional)

- For advanced deployments, use Kubernetes to orchestrate and manage your containers. Create Deployment and Service YAML files for each microservice.

Example Kubernetes Deployment for Product Service:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
      - name: product-service
        image: productservice:latest
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: product-service
spec:
  selector:
    app: product-service
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
```

10. Security

- Implement robust authentication and authorization mechanisms, and ensure secure communication between services using HTTPS.

VII. CONCLUSION

Microservices architecture offers significant advantages in terms of scalability, agility, and maintainability. However, it also introduces complexities related to distributed systems, testing, and data management. By adhering to core principles and best practices, developers can harness the benefits of microservices while mitigating the challenges. Future trends in microservices include increased use of serverless architectures, improved observability tools, and AI-powered deployment automation. The use of C# with ASP.NET Core provides a robust and well-supported platform for building microservices-based applications, leveraging the strength of the .NET ecosystem.

VIII. REFERENCES

[1] Lewis, J.; Fowler, M. Microservices—A Definition of This New Architectural Term; EA PAD: Redwood City, CA, USA, 2014. [Google Scholar]

[2] Merson, P. Microservices beyond the Hype: What You Gain and What You Lose; SEI Digital Library: San Diego, CA, USA, 2015. [Google Scholar].

[3] Microservices Examples: Amazon, Netflix, Uber, and Etsy - Blog. (2024). https://blog.dreamfactory.com/microservices-examples.